Cavli C16QS Integration with ESP32 in ESP-IDF

Manual AT command setup, AT+PPPSTART, lwIP PPPoS, DNS to IP, and HTTP POST using BSD sockets

This PDF describes only the ESP-IDF integration path. It follows the same concept as the STM32 reference flow: prepare the modem in AT mode, put the Cavli module into PPP mode using AT+PPPSTART, then send data through IP sockets from the ESP32.

1. Target flow

- ESP32 talks to Cavli C16QS over UART in normal AT command mode.
- ESP32 sends the modem setup commands: AT, ATE0, SIM check, network check, APN, PDP activate.
- ESP32 sends AT+PPPSTART. After this point the UART becomes PPP binary data, not AT command text.
- ESP-IDF/lwIP PPPoS runs over the same UART and obtains an IP address.
- ESP32 resolves webhook.site to an IP address using DNS.
- ESP32 opens a TCP socket to that IP on port 80 and sends POST body: shuva.
- ESP32 prints the full HTTP response on the serial monitor.

2. Hardware wiring

Use a common ground and cross the UART lines. The default pins in the example are GPIO17 for ESP32 TX and GPIO16 for ESP32 RX.

ESP32	Cavli C16QS	Note
GPIO17 / UART TX	C16QS RX	ESP32 sends AT and PPP data to modem
GPIO16 / UART RX	C16QS TX	ESP32 receives modem and PPP data
GND	GND	Must be common
Power	C16QS supply	Use a strong cellular modem supply

Important: many cellular modules draw high current pulses. A weak supply can cause random resets, PPP negotiation failures, or intermittent socket failures.

3. ESP-IDF project files

Folder structure

```
cavli_ppp_post/
|-- CMakeLists.txt
`-- main/
    |-- CMakeLists.txt
    `-- main.c
```

Top-level CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)

include($ENV{IDF_PATH}/tools/cmake/project.cmake)
project(cavli_ppp_post)
```

main/CMakeLists.txt

For ESP-IDF 5.3, 5.4, 6.x and newer:

```
idf_component_register(
    SRCS "main.c"
    INCLUDE_DIRS "."
    REQUIRES esp_driver_uart esp_netif esp_event lwip
```

```
)
```

For older ESP-IDF versions, if esp_driver_uart is not found, use driver instead:

```
idf_component_register(
    SRCS "main.c"
    INCLUDE_DIRS "."
    REQUIRES driver esp_netif esp_event lwip
)
```

4. menuconfig settings

Run this first:

```
idf.py menuconfig
```

Then open:

```
Component config
`-- LWIP
```

Enable these settings:

```
[*] Enable PPP support
[*] Enable IPV4 support for PPP connections (IPCP)
```

Recommended while debugging PPP:

```
[*] Enable PPP debug log output
[*] Enable Notify Phase Callback
```

If PPP negotiation gets stuck around IPv6CP, disable IPv6 PPP:

```
[ ] Enable IPV6 support for PPP connections (IPV6CP)
```

Optional sdkconfig.defaults:

```
CONFIG_LWIP_PPP_SUPPORT=y
CONFIG_LWIP_PPP_ENABLE_IPV4=y
CONFIG_LWIP_PPP_ENABLE_IPV6=n
CONFIG_LWIP_PPP_DEBUG_ON=y
CONFIG_LWIP_PPP_NOTIFY_PHASE_SUPPORT=y
```

5. Packages or components needed

- No external component is required for this manual PPPoS example.
- The code uses ESP-IDF built-in components: UART driver, esp_netif, esp_event, lwIP, FreeRTOS, and BSD sockets.
- Do not add esp_modem for this version. This guide intentionally sends the Cavli AT sequence manually before starting PPPoS.
- Recommended ESP-IDF: v5.3 or newer. Older versions can work, but CMake component names may differ.

6. Full main.c

Update MODEM_TX_PIN, MODEM_RX_PIN, MODEM_APN, WEBHOOK_HOST, WEBHOOK_PATH, and POST_BODY as needed.

```
#include
#include
#include
#include

#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/event_groups.h"

#include "esp_log.h"
#include "esp_err.h"
#include "esp_timer.h"
#include "esp_event.h"
#include "esp_netif.h"
#include "esp_idf_version.h"

#include "driver/uart.h"
```

```
#include "lwip/netif.h"
#include "lwip/ip4_addr.h"
#include "lwip/ip_addr.h"
#include "lwip/dns.h"
#include "lwip/sockets.h"
#include "lwip/netdb.h"
#include "lwip/inet.h"

#include "netif/ppp/pppos.h"
#include "netif/ppp/pppapi.h"

#define MODEM_UART_NUM UART_NUM_2

// ESP32 TX -> Cavli RX
// ESP32 RX <- Cavli TX
#define MODEM_TX_PIN 17
#define MODEM_RX_PIN 16
#define MODEM_BAUD   115200

#define MODEM_APN "airtelgprs.com"

#define WEBHOOK_HOST "webhook.site"
#define WEBHOOK_PATH "/dc7f2110-9875-463e-8743-9a91ee95cd2a"
#define WEBHOOK_PORT 80

#define POST_BODY "shuva"

#define MODEM_AT_RESP_SIZE          512
#define MODEM_NETWORK_TIMEOUT_MS    180000
#define MODEM_NETWORK_RETRY_MS      5000
#define MODEM_PPPSTART_TIMEOUT_MS   15000
#define PPP_CONNECT_TIMEOUT_MS      60000

#define PPP_CONNECTED_BIT BIT0
#define PPP_FAILED_BIT    BIT1

static const char *TAG = "CAVLI_PPP";

static ppp_pcb *g_ppp = NULL;
static struct netif g_ppp_netif;
static EventGroupHandle_t g_ppp_event_group = NULL;

static uint32_t get_ms(void)
{
    return (uint32_t)(esp_timer_get_time() / 1000ULL);
}

static void modem_uart_init(void)
{
    uart_config_t uart_config = {
        .baud_rate = MODEM_BAUD,
        .data_bits = UART_DATA_8_BITS,
        .parity = UART_PARITY_DISABLE,
        .stop_bits = UART_STOP_BITS_1,
        .flow_ctrl = UART_HW_FLOWCTRL_DISABLE,
#if ESP_IDF_VERSION_MAJOR >= 5
        .source_clk = UART_SCLK_DEFAULT,
#endif
    };

    ESP_ERROR_CHECK(uart_driver_install(MODEM_UART_NUM, 4096, 4096, 0, NULL, 0));
    ESP_ERROR_CHECK(uart_param_config(MODEM_UART_NUM, &uart;_config));
    ESP_ERROR_CHECK(uart_set_pin(
        MODEM_UART_NUM,
        MODEM_TX_PIN,
        MODEM_RX_PIN,
        UART_PIN_NO_CHANGE,
        UART_PIN_NO_CHANGE
    ));
    ESP_ERROR_CHECK(uart_flush_input(MODEM_UART_NUM));
}

static void modem_flush_rx(uint32_t timeout_ms)
{
    uint8_t buf[128];
    uint32_t start = get_ms();

    while ((get_ms() - start) < timeout_ms)
    {
        int len = uart_read_bytes(MODEM_UART_NUM, buf, sizeof(buf), pdMS_TO_TICKS(10));
        if (len <= 0)
        {
            vTaskDelay(pdMS_TO_TICKS(1));
        }
    }
}

static bool modem_wait_response(char *resp,
                                size_t resp_size,
                                uint32_t timeout_ms,
```

```
                              const char *token1,
                              const char *token2,
                              const char *token3)
{
    uint32_t start = get_ms();
    size_t resp_len = 0;

    if (resp && resp_size > 0)
    {
        resp[0] = '\0';
    }

    while ((get_ms() - start) < timeout_ms)
    {
        uint8_t ch;
        int len = uart_read_bytes(MODEM_UART_NUM, &ch;, 1, pdMS_TO_TICKS(20));

        if (len == 1)
        {
            putchar((char)ch);

            if (resp && resp_size > 1 && resp_len < (resp_size - 1))
            {
                resp[resp_len++] = (char)ch;
                resp[resp_len] = '\0';
            }

            if (resp)
            {
                if (token1 && strstr(resp, token1)) return true;
                if (token2 && strstr(resp, token2)) return true;
                if (token3 && strstr(resp, token3)) return true;
            }
        }
    }

    return false;
}

static bool modem_send_at(const char *cmd,
                          char *resp,
                          size_t resp_size,
                          uint32_t timeout_ms)
{
    modem_flush_rx(50);

    ESP_LOGI(TAG, "MODEM >>> %s", cmd);

    uart_write_bytes(MODEM_UART_NUM, cmd, strlen(cmd));
    uart_write_bytes(MODEM_UART_NUM, "\r", 1);
    uart_wait_tx_done(MODEM_UART_NUM, pdMS_TO_TICKS(1000));

    bool got_token = modem_wait_response(resp, resp_size, timeout_ms, "OK", "ERROR", NULL);

    if (!got_token)
    {
        ESP_LOGE(TAG, "Timeout after command: %s", cmd);
        return false;
    }

    if (resp && strstr(resp, "OK"))
    {
        ESP_LOGI(TAG, "MODEM command OK");
        return true;
    }

    ESP_LOGE(TAG, "MODEM command ERROR: %s", cmd);
    return false;
}

static bool modem_response_has_reg(const char *resp)
{
    if (resp == NULL)
    {
        return false;
    }

    // ,1 = home network. ,5 = roaming network.
    return strstr(resp, ",1") || strstr(resp, ",5");
}

static bool modem_wait_network(uint32_t timeout_ms)
{
    char csq[MODEM_AT_RESP_SIZE];
    char creg[MODEM_AT_RESP_SIZE];
    char cereg[MODEM_AT_RESP_SIZE];
    char cgatt[MODEM_AT_RESP_SIZE];
    char tmp[MODEM_AT_RESP_SIZE];

    uint32_t start = get_ms();
```

```
    ESP_LOGI(TAG, "Waiting for cellular network registration...");

    while ((get_ms() - start) < timeout_ms)
    {
        modem_send_at("AT+CSQ", csq, sizeof(csq), 3000);
        modem_send_at("AT+CREG?", creg, sizeof(creg), 3000);
        modem_send_at("AT+CEREG?", cereg, sizeof(cereg), 3000);
        modem_send_at("AT+CGATT?", cgatt, sizeof(cgatt), 3000);

        bool registered_ok = modem_response_has_reg(creg) || modem_response_has_reg(cereg);
        bool attached_ok = strstr(cgatt, "+CGATT: 1") != NULL;

        if (registered_ok && attached_ok)
        {
            ESP_LOGI(TAG, "Network ready");
            return true;
        }

        if (registered_ok && !attached_ok)
        {
            ESP_LOGW(TAG, "Registered but not attached. Sending AT+CGATT=1");
            modem_send_at("AT+CGATT=1", tmp, sizeof(tmp), 15000);
        }

        ESP_LOGW(TAG, "Network not ready. Retrying in 5 seconds...");
        vTaskDelay(pdMS_TO_TICKS(MODEM_NETWORK_RETRY_MS));
    }

    ESP_LOGE(TAG, "Network registration timeout");
    return false;
}

static bool modem_enter_ppp_mode(void)
{
    char resp[MODEM_AT_RESP_SIZE];
    char cmd[128];

    ESP_LOGI(TAG, "Preparing Cavli modem for PPP mode");

    if (!modem_send_at("AT", resp, sizeof(resp), 3000)) return false;
    if (!modem_send_at("ATE0", resp, sizeof(resp), 3000)) return false;
    if (!modem_send_at("AT+CMEE=2", resp, sizeof(resp), 3000)) return false;

    if (!modem_send_at("AT+CPIN?", resp, sizeof(resp), 3000)) return false;

    if (strstr(resp, "READY") == NULL)
    {
        ESP_LOGE(TAG, "SIM not ready");
        return false;
    }

    if (!modem_send_at("AT+CFUN=1", resp, sizeof(resp), 15000)) return false;

    // Auto operator selection. If this is slow on your SIM, it is still normal.
    modem_send_at("AT+COPS=0", resp, sizeof(resp), 60000);

    modem_send_at("AT+CREG=1", resp, sizeof(resp), 3000);
    modem_send_at("AT+CEREG=1", resp, sizeof(resp), 3000);

    if (!modem_wait_network(MODEM_NETWORK_TIMEOUT_MS))
    {
        return false;
    }

    snprintf(cmd, sizeof(cmd), "AT+CGDCONT=1,\"IP\",\"%s\"", MODEM_APN);

    if (!modem_send_at(cmd, resp, sizeof(resp), 5000))
    {
        return false;
    }

    modem_send_at("AT+CGDCONT?", resp, sizeof(resp), 5000);

    if (!modem_send_at("AT+CGACT=1,1", resp, sizeof(resp), 30000))
    {
        ESP_LOGE(TAG, "AT+CGACT=1,1 failed");
        return false;
    }

    modem_send_at("AT+CGACT?", resp, sizeof(resp), 5000);

    modem_flush_rx(100);

    ESP_LOGI(TAG, "MODEM >>> AT+PPPSTART");

    uart_write_bytes(MODEM_UART_NUM, "AT+PPPSTART\r", strlen("AT+PPPSTART\r"));
    uart_wait_tx_done(MODEM_UART_NUM, pdMS_TO_TICKS(1000));

    bool pppstart_ok = modem_wait_response(
```

```
        resp,
        sizeof(resp),
        MODEM_PPPSTART_TIMEOUT_MS,
        "CONNECT",
        "+PPPSTART",
        "ERROR"
    );

    if (!pppstart_ok)
    {
        ESP_LOGE(TAG, "AT+PPPSTART timeout");
        return false;
    }

    if (strstr(resp, "CONNECT") || strstr(resp, "+PPPSTART"))
    {
        ESP_LOGI(TAG, "PPP mode started");
        ESP_LOGI(TAG, "UART is now PPP binary mode. Do not send more AT commands.");
        return true;
    }

    ESP_LOGE(TAG, "AT+PPPSTART failed");
    return false;
}

static u32_t ppp_output_cb(ppp_pcb *pcb, u8_t *data, u32_t len, void *ctx)
{
    (void)pcb;
    (void)ctx;

    int written = uart_write_bytes(MODEM_UART_NUM, (const char *)data, len);

    if (written < 0)
    {
        return 0;
    }

    return (u32_t)written;
}

static void ppp_status_cb(ppp_pcb *pcb, int err_code, void *ctx)
{
    (void)ctx;

    struct netif *pppif = ppp_netif(pcb);

    switch (err_code)
    {
        case PPPERR_NONE:
        {
            ESP_LOGI(TAG, "PPP CONNECTED");
            ESP_LOGI(TAG, "PPP IP  : %s", ip4addr_ntoa(netif_ip4_addr(pppif)));
            ESP_LOGI(TAG, "PPP GW  : %s", ip4addr_ntoa(netif_ip4_gw(pppif)));
            ESP_LOGI(TAG, "PPP MASK: %s", ip4addr_ntoa(netif_ip4_netmask(pppif)));

            // Force known DNS servers. Remove this if peer DNS works correctly.
            ip_addr_t dns1;
            ip_addr_t dns2;

            IP_ADDR4(&dns1;, 8, 8, 8, 8);
            IP_ADDR4(&dns2;, 1, 1, 1, 1);

            dns_setserver(0, &dns1;);
            dns_setserver(1, &dns2;);

            ESP_LOGI(TAG, "DNS0: %s", ipaddr_ntoa(dns_getserver(0)));
            ESP_LOGI(TAG, "DNS1: %s", ipaddr_ntoa(dns_getserver(1)));

            xEventGroupSetBits(g_ppp_event_group, PPP_CONNECTED_BIT);
            break;
        }

        case PPPERR_CONNECT:
            ESP_LOGE(TAG, "PPPERR_CONNECT: connection lost");
            xEventGroupSetBits(g_ppp_event_group, PPP_FAILED_BIT);
            break;

        case PPPERR_AUTHFAIL:
            ESP_LOGE(TAG, "PPPERR_AUTHFAIL");
            xEventGroupSetBits(g_ppp_event_group, PPP_FAILED_BIT);
            break;

        case PPPERR_PROTOCOL:
            ESP_LOGE(TAG, "PPPERR_PROTOCOL");
            xEventGroupSetBits(g_ppp_event_group, PPP_FAILED_BIT);
            break;

        case PPPERR_PEERDEAD:
            ESP_LOGE(TAG, "PPPERR_PEERDEAD");
            xEventGroupSetBits(g_ppp_event_group, PPP_FAILED_BIT);
```

```
            break;

        default:
            ESP_LOGE(TAG, "PPP error code: %d", err_code);
            xEventGroupSetBits(g_ppp_event_group, PPP_FAILED_BIT);
            break;
    }
}

static void ppp_rx_task(void *arg)
{
    (void)arg;

    uint8_t rx_buf[256];

    ESP_LOGI(TAG, "PPP RX task started");

    while (1)
    {
        int len = uart_read_bytes(MODEM_UART_NUM, rx_buf, sizeof(rx_buf), pdMS_TO_TICKS(20));

        if (len > 0 && g_ppp != NULL)
        {
            pppos_input_tcpip(g_ppp, rx_buf, len);
        }
    }
}

static bool start_pppos(void)
{
    ESP_LOGI(TAG, "Starting lwIP PPPoS");

    g_ppp_event_group = xEventGroupCreate();

    if (g_ppp_event_group == NULL)
    {
        ESP_LOGE(TAG, "Failed to create PPP event group");
        return false;
    }

    g_ppp = pppos_create(&g;_ppp_netif, ppp_output_cb, ppp_status_cb, NULL);

    if (g_ppp == NULL)
    {
        ESP_LOGE(TAG, "pppos_create failed");
        return false;
    }

    ppp_set_default(g_ppp);
    ppp_set_usepeerdns(g_ppp, 1);

    BaseType_t task_ok = xTaskCreate(
        ppp_rx_task,
        "ppp_rx_task",
        4096,
        NULL,
        18,
        NULL
    );

    if (task_ok != pdPASS)
    {
        ESP_LOGE(TAG, "Failed to create PPP RX task");
        return false;
    }

    ESP_LOGI(TAG, "Starting PPP negotiation");
    ppp_connect(g_ppp, 0);

    EventBits_t bits = xEventGroupWaitBits(
        g_ppp_event_group,
        PPP_CONNECTED_BIT | PPP_FAILED_BIT,
        pdFALSE,
        pdFALSE,
        pdMS_TO_TICKS(PPP_CONNECT_TIMEOUT_MS)
    );

    if (bits & PPP_CONNECTED_BIT)
    {
        ESP_LOGI(TAG, "PPP connected successfully");
        return true;
    }

    ESP_LOGE(TAG, "PPP connect timeout or failed");
    return false;
}

static bool socket_send_all(int sock, const char *data, size_t len)
{
    size_t sent = 0;
```

```
    while (sent < len)
    {
        int n = send(sock, data + sent, len - sent, 0);

        if (n <= 0)
        {
            ESP_LOGE(TAG, "send failed, errno=%d", errno);
            return false;
        }

        sent += (size_t)n;
    }

    return true;
}

static bool http_post_to_webhook_ip(void)
{
    struct addrinfo hints;
    struct addrinfo *res = NULL;
    struct sockaddr_in server_addr;

    memset(&hints;, 0, sizeof(hints));
    hints.ai_family = AF_INET;
    hints.ai_socktype = SOCK_STREAM;

    ESP_LOGI(TAG, "Resolving %s to IP", WEBHOOK_HOST);

    int dns_ret = getaddrinfo(WEBHOOK_HOST, NULL, &hints;, &res;);

    if (dns_ret != 0 || res == NULL)
    {
        ESP_LOGE(TAG, "DNS failed. getaddrinfo ret=%d", dns_ret);
        return false;
    }

    memcpy(&server;_addr, res->ai_addr, sizeof(struct sockaddr_in));
    server_addr.sin_port = htons(WEBHOOK_PORT);

    ESP_LOGI(TAG, "%s resolved IP: %s", WEBHOOK_HOST, inet_ntoa(server_addr.sin_addr));

    freeaddrinfo(res);

    int sock = socket(AF_INET, SOCK_STREAM, IPPROTO_IP);

    if (sock < 0)
    {
        ESP_LOGE(TAG, "socket create failed, errno=%d", errno);
        return false;
    }

    struct timeval timeout;
    timeout.tv_sec = 15;
    timeout.tv_usec = 0;

    setsockopt(sock, SOL_SOCKET, SO_RCVTIMEO, &timeout;, sizeof(timeout));
    setsockopt(sock, SOL_SOCKET, SO_SNDTIMEO, &timeout;, sizeof(timeout));

    ESP_LOGI(TAG, "Connecting to IP %s:%d", inet_ntoa(server_addr.sin_addr), WEBHOOK_PORT);

    if (connect(sock, (struct sockaddr *)&server;_addr, sizeof(server_addr)) != 0)
    {
        ESP_LOGE(TAG, "connect failed, errno=%d", errno);
        close(sock);
        return false;
    }

    ESP_LOGI(TAG, "TCP connected");

    char request[512];

    int request_len = snprintf(
        request,
        sizeof(request),
        "POST %s HTTP/1.1\r\n"
        "Host: %s\r\n"
        "User-Agent: ESP32-Cavli-PPPoS/1.0\r\n"
        "Content-Type: text/plain\r\n"
        "Content-Length: %u\r\n"
        "Connection: close\r\n"
        "\r\n"
        "%s",
        WEBHOOK_PATH,
        WEBHOOK_HOST,
        (unsigned int)strlen(POST_BODY),
        POST_BODY
    );

    if (request_len <= 0 || request_len >= sizeof(request))
```

```
    {
        ESP_LOGE(TAG, "HTTP request buffer too small");
        close(sock);
        return false;
    }

    ESP_LOGI(TAG, "Sending HTTP POST");
    printf("\n---------------- HTTP REQUEST ----------------\n");
    printf("%s\n", request);
    printf("----------------------------------------------\n");

    if (!socket_send_all(sock, request, request_len))
    {
        ESP_LOGE(TAG, "HTTP send failed");
        close(sock);
        return false;
    }

    ESP_LOGI(TAG, "HTTP request sent. Reading response...");
    printf("\n---------------- HTTP RESPONSE ---------------\n");

    char rx[512];

    while (1)
    {
        int len = recv(sock, rx, sizeof(rx) - 1, 0);

        if (len <= 0)
        {
            break;
        }

        rx[len] = '\0';
        printf("%s", rx);
    }

    printf("\n----------------------------------------------\n");

    close(sock);

    ESP_LOGI(TAG, "HTTP POST done");
    return true;
}

void app_main(void)
{
    ESP_LOGI(TAG, "ESP32 + Cavli C16QS AT+PPPSTART + PPPoS + HTTP POST");

    ESP_ERROR_CHECK(esp_netif_init());
    ESP_ERROR_CHECK(esp_event_loop_create_default());

    modem_uart_init();

    if (!modem_enter_ppp_mode())
    {
        ESP_LOGE(TAG, "Failed to enter PPP mode");
        return;
    }

    if (!start_pppos())
    {
        ESP_LOGE(TAG, "Failed to start PPPoS");
        return;
    }

    if (!http_post_to_webhook_ip())
    {
        ESP_LOGE(TAG, "First HTTP POST failed");
        return;
    }

    ESP_LOGI(TAG, "First HTTP POST successful");

    while (1)
    {
        vTaskDelay(pdMS_TO_TICKS(30000));
        http_post_to_webhook_ip();
    }
}
```

7. Build, flash, and monitor

```
idf.py set-target esp32
idf.py menuconfig
idf.py build
idf.py flash monitor
```

For ESP32-S3, use:

```
idf.py set-target esp32s3
```

8. What successful output should look like

```
MODEM >>> AT
OK
MODEM >>> AT+CPIN?
+CPIN: READY
OK
MODEM >>> AT+CGDCONT=1,"IP","airtelgprs.com"
OK
MODEM >>> AT+CGACT=1,1
OK
MODEM >>> AT+PPPSTART
CONNECT
PPP CONNECTED
PPP IP  : xxx.xxx.xxx.xxx
Resolving webhook.site to IP
webhook.site resolved IP: xxx.xxx.xxx.xxx
TCP connected
HTTP RESPONSE
HTTP/1.1 200 OK
...
```

9. Debugging checklist

- No response to AT: check crossed TX/RX, common GND, baud rate, and modem power.
- SIM not ready: check SIM orientation, SIM activation, PIN lock, and antenna.
- Not registered: check antenna, signal strength with AT+CSQ, operator support, and network coverage.
- AT+CGACT fails: check APN and SIM data plan. For Airtel India, airtelgprs.com is commonly used.
- AT+PPPSTART works but PPP never connects: enable PPP debug logs, disable PPP IPv6CP, and check UART data is not being printed or modified.
- DNS fails but PPP has IP: keep the forced DNS lines in ppp_status_cb or try another DNS server.
- TCP connects but webhook does not receive data: keep Host: webhook.site in the HTTP request even when connecting to the IP.
- HTTPS URL will not work with this plain socket example. Use http:// and port 80, or add TLS for port 443.

10. Important notes

- After AT+PPPSTART succeeds, do not send AT commands on that UART. The UART is now carrying PPP frames.
- The code resolves webhook.site to an IP before connecting, but HTTP still requires the Host header for correct routing.
- This is a plain HTTP example. For HTTPS you must use TLS after PPP is connected.
- If you already use another ESP32 network interface such as Wi-Fi, be careful with DNS because lwIP DNS servers are global.

11. Reference points

This guide is based on the user's STM32 PPP reference flow and current ESP-IDF documentation. ESP-IDF documentation states that lwIP supports BSD sockets, DNS, and PPPoS for modem interfaces, and the UART driver is configured by installing the driver, setting parameters, and assigning pins.

```
Official ESP-IDF lwIP guide:
https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/lwip.html

Official ESP-IDF UART driver guide:
https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/uart.html

Official ESP-IDF project configuration guide:
https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-guides/kconfig/project-
    configuration-guide.html
```